// master-doc v1.0

TOASTY KEY

The API cost layer for AI-native builders.

Track. Control. Understand.

Product Specification + Technical Architecture + Branding Guide

April 2026 · Confidential · Toasty Media

Contents

01 The Problem 3
02 The Solution: ToastyKey 4
03 Feature Specification 5
04 Technical Architecture 9
05 MCP Integration 11
06 Dashboard Design 12
07 Competitor Analysis 14
08 Branding Guide 15
09 GitHub Strategy 16
10 Build Plan 17

01. The Problem

AI made building software easy. But managing the cost of what you built is still a nightmare.

A typical AI-native project in 2026 touches 5–15 different APIs: LLM inference (Claude, GPT, Gemini), image generation (Flux, DALL-E), voice synthesis (ElevenLabs, Cartesia), video generation (Runway, Kling), hosting (Vercel, Railway), databases, authentication services, SMS providers, and more. Each one requires a separate account, separate billing, separate usage dashboard, and separate mental model for pricing.

For non-technical builders using Claude Code, Cursor, or other AI coding tools, this creates four specific pain points:

Pain 1: Cost Blindness

You have no idea what your project actually costs to run. OpenAI charges per 1K tokens. ElevenLabs charges per character. Runway charges per second. Vercel charges per function invocation. There is no unified view. Most builders discover their actual costs when the credit card bill arrives — or when an API cuts them off.

Pain 2: Runaway Agents

AI agents can enter infinite loops. A broken script can call an API 10,000 times in minutes. A misconfigured workflow can drain $500 overnight. There is no safety net. No kill switch. No anomaly detection. You wake up to an empty balance and a broken project.

Pain 3: Key Chaos

API keys scattered across .env files in 6 different project directories. Keys accidentally pushed to GitHub. Keys that expired and broke production. Keys you forgot you had. No central place to see what you have, what’s active, what’s costing you money.

Pain 4: Zero Learning Loop

After months of building, you cannot answer basic questions: Which project costs the most? Which API provider gives you the best value? Is your spending going up or down? What does each feature of your app actually cost to run? No tool connects your development activity to your financial reality.

> *The infrastructure for building with AI exists. The infrastructure for understanding what it costs does not.*

02. The Solution: ToastyKey

ToastyKey is an MCP server that acts as the cost intelligence layer for AI-native development. It runs locally on your machine, sits between your code and your API providers, and gives you complete visibility and control over every API call, every dollar spent, and every key in your ecosystem.

> *One-liner: ToastyKey is the layer between your AI code and your real-world cost.*

How It Works (30-Second Version)

1. Install ToastyKey. It runs as a local MCP server on your machine.
2. Store your API keys in ToastyKey instead of .env files.
3. Your code calls APIs through ToastyKey's proxy (one-line change per project).
4. ToastyKey logs every call, tracks every cost, and shows you a real-time dashboard on localhost.
5. Set budget caps, anomaly triggers, and project-level tracking. ToastyKey handles the rest.
6. Claude Code connects to ToastyKey via MCP, so you can check costs without leaving your coding session.

What ToastyKey Is Not

It is not a payment aggregator. It does not hold your money or pay your API bills. It is not a cloud service — it runs entirely on your machine. It does not require you to change your API providers. It is not an agent framework or orchestrator. It is a monitoring and control layer. Simple, local, powerful.

03. Feature Specification

3.1 API Cost Tracker

Every API call that passes through ToastyKey is logged with: provider name, endpoint called, request size (tokens/characters/seconds depending on provider), response size, cost calculated in real-time, timestamp, which project made the call, and which Claude Code session it originated from.

Dual-View Display

The dashboard shows costs in two formats simultaneously:

Raw view: The actual API pricing — “$0.003 per 1K input tokens, $0.015 per 1K output tokens.” This is for developers who want precision.

Tangible output view: Human-readable translation — “You generated 47 images today (₹182), synthesised 12 minutes of audio (₹340), and made 230 LLM calls (₹127).” This is for builders who want to understand what they actually got for their money.

Metrics Tracked

Metric	Description	Granularity
Total spend	Cumulative cost across all providers	Daily / weekly / monthly / all-time
Spend per provider	Cost breakdown by API provider	Per provider, per day
Spend per project	Cost attributed to each project directory	Per project, per session
Spend per session	Cost of each Claude Code / Cursor session	Per session
Cost per API call (avg)	Average cost per individual call	Per provider, per project
Token usage	Input/output tokens for LLM providers	Per call, per session, per day
Character usage	Characters sent for TTS/voice providers	Per call, per day
Generation count	Images, videos, audio files generated	Per provider, per day
API response time (avg)	Latency per provider per endpoint	Per provider, per hour
API reliability rate	Success/failure ratio per provider	Per provider, trailing 7 days
Cost velocity	How fast you are spending ($/hour)	Real-time
Projected monthly cost	Extrapolation of current usage	Updated daily
Cost trend	Spending compared to previous period	Week-over-week, month-over-month
Most expensive call	Single highest-cost API call logged	All-time, per project

Cheapest alternative	If a cheaper model could have handled the call	Per LLM call (advisory)

3.2 Budget Caps & Spending Controls

Users can set spending limits at multiple levels. Each level can be configured independently.

Level	Example	Behaviour When Hit
Global daily cap	“Don’t spend more than ₹500 today”	Blocks all API calls until next day
Global weekly cap	“Don’t exceed ₹2,000 this week”	Blocks all calls until next week
Global monthly cap	“Max ₹8,000 this month”	Hard stop until manual override
Per-provider cap	“OpenAI max ₹300/day”	Blocks only that provider
Per-project cap	“Spazi project max ₹1,000 total”	Blocks calls from that project
Per-session cap	“This coding session max ₹200”	Blocks calls from that session
Soft warning	“Alert me at 80% of any cap”	Dashboard notification + Claude Code alert

When a cap is hit, the user receives an immediate notification in the dashboard and (if in Claude Code) via the MCP connection. The user can choose to override, increase the cap, or stop. No API call is made without explicit permission once a cap is reached.

3.3 Anomaly Detection & Triggers

Anomaly detection monitors API call patterns and fires triggers when something looks wrong. Users configure triggers at the level of granularity they need.

Trigger Configuration Levels

Global: applies to all APIs across all projects. Per-provider: applies to a specific API (e.g., only ElevenLabs). Per-project: applies to a specific project directory. Per-API-endpoint: applies to a specific endpoint (e.g., only /v1/chat/completions).

Trigger Types

Trigger	Description	Default Threshold
Rate spike	Calls per minute exceeds Nx normal	5x baseline in 2-minute window
Cost spike	Spend per hour exceeds Nx normal	3x average hourly spend
Error storm	Error rate exceeds threshold	>50% errors in 20+ calls
Token explosion	Single call uses abnormal tokens	>10x average token count
Silent drain	Steady unexplained background usage	Calls when no active session
New provider call	First call to a provider not seen before	Any new provider domain

Trigger Actions (User-Configurable)

For each trigger, users choose the response:

Log only: Record the event, take no action. Dashboard notification: Show an alert on the dashboard. Claude Code alert: Push notification to active Claude Code session via MCP. Auto-pause: Block further calls from the source and wait for user confirmation. Auto-kill: Immediately terminate all API calls from that source. Custom webhook: Send alert to a URL (for future Slack/WhatsApp integration).

3.4 Project-Level Tracking

ToastyKey automatically detects which project directory initiated an API call and attributes the cost accordingly. This works by reading the current working directory when a call is proxied.

Project Dashboard Metrics

Total cost (lifetime and per-period). Cost breakdown by provider within the project. Number of API calls. Average cost per call. Cost trend over time. Most expensive day. Active sessions count. Time since last API call.

Session Tracking

Each Claude Code session (terminal or desktop) that connects to ToastyKey is tracked as a distinct session. Users can see: session start time, session duration, total cost for that session, APIs used, and calls made. This includes Claude Code's own Bedrock/Anthropic API usage if the user configures ToastyKey to monitor it.

3.5 Key Vault

A centralised, encrypted store for all API keys. Keys are stored locally on the user's machine with AES-256 encryption. Never transmitted externally.

Key Vault Features

Add/remove/edit keys with provider name and label. Keys are hidden by default — click to reveal (auto-hides after 10 seconds). Copy to clipboard with one click. Key health status: active (last successful call), expired (last call failed with auth error), unused (no calls in 30+ days). Key creation date and last used date. Associated projects (which projects use this key). Monthly cost attributed to this key. Rotate key: update in one place, propagated to all projects. Export key list (encrypted) for backup. Import from .env file (parse and store automatically).

Key Vault Metrics

Total active keys. Keys by provider. Most expensive key (by monthly cost). Keys approaching rate limits. Keys with high error rates. Unused keys (candidates for cleanup).

3.6 Usage Reports

Automated reports generated on a configurable schedule (daily, weekly, monthly). Reports include:

Summary: Total spend, change from last period, top provider, top project. Breakdown: Spend per provider, spend per project, spend per day. Trends: Cost velocity changes, usage pattern shifts. Anomalies: Any triggers fired, what caused them, what action was taken.

Recommendations: Providers with high error rates (consider switching), unused keys (consider removing), projects with rising costs (investigate), cheaper model alternatives for specific use cases.

Reports are saved locally as HTML files and accessible from the dashboard. Can be exported as PDF.

04. Technical Architecture

4.1 System Overview

ToastyKey runs as a local service on the user's machine. It consists of three components: a proxy server, a dashboard server, and an MCP server. All three share one SQLite database.

Data Flow

Your code → ToastyKey proxy (localhost:4000) → logs the request → checks budget caps → if approved, forwards to the real API provider → receives response → logs response + cost → returns response to your code. Simultaneously, the dashboard (localhost:3000) reads from the database and displays real-time metrics. Claude Code connects to the MCP server and can query costs, set budgets, and receive alerts inline.

4.2 Tech Stack

Component	Technology	Rationale
Proxy server	Node.js + Express	Lightweight, fast, handles concurrent requests. You already work with Node.
Database	SQLite (via better-sqlite3)	Zero setup, no external dependencies, runs locally, fast for read-heavy workloads.
Dashboard	React + Vite	Fast dev, hot reload, modern. Served from localhost:3000.
Charts	Recharts	Simple, React-native, handles all chart types needed.
MCP server	Node.js (@modelcontextprotocol/sdk)	Official MCP SDK. Claude Code connects natively.
Encryption	Node.js crypto (AES-256-GCM)	For key vault. Industry-standard local encryption.
Process manager	Node.js (single process)	One npm start command launches proxy + dashboard + MCP.
Cost calculation	Local pricing database (JSON)	Provider pricing stored locally, updated periodically via GitHub.

4.3 Database Schema (Core Tables)

Table	Key Fields	Purpose
api_calls	id, timestamp, provider, endpoint, project, session_id, input_tokens, output_tokens, cost_usd, cost_inr, status, latency_ms	Every API call logged

api_keys	id, provider, label, encrypted_key, created_at, last_used, status, total_cost	Key vault storage
projects	id, name, directory_path, total_cost, created_at	Project registry
sessions	id, project_id, tool (claude_code/cursor/other), started_at, ended_at, total_cost	Session tracking
budgets	id, scope (global/provider/project), scope_id, period (daily/weekly/monthly), limit_amount, current_spend	Budget configuration
triggers	id, scope, scope_id, trigger_type, threshold, action, enabled	Anomaly trigger configuration
trigger_events	id, trigger_id, timestamp, details, action_taken	Trigger history
reports	id, period, generated_at, html_content, summary_json	Generated reports

4.4 Proxy Architecture

The proxy intercepts outgoing API calls by providing a local endpoint that mirrors each supported provider's API. The user's code points to localhost:4000/openai/v1/chat/completions instead of api.openai.com/v1/chat/completions. ToastyKey forwards the request with the real API key from the vault, logs everything, and returns the response unchanged.

For providers where direct proxy isn't practical, ToastyKey offers a middleware SDK: a one-line wrapper that intercepts calls made through the provider's official SDK and logs them to ToastyKey without changing the API endpoint.

Supported Providers (v1)

OpenAI (GPT-4o, GPT-4o-mini, DALL-E, Whisper). Anthropic (Claude via direct API). AWS Bedrock (Claude via Bedrock). ElevenLabs (voice synthesis). Cartesia (TTS). Replicate (image/video models). Stability AI (image generation). Generic REST API (catch-all with manual cost configuration).

Claude Code / Bedrock Usage Tracking

ToastyKey can monitor Claude Code's own API usage by reading Bedrock CloudWatch metrics (if configured) or by parsing Claude Code's local usage logs. This gives users total development cost visibility — not just the APIs their projects call, but the AI tool they're coding with.

05. MCP Integration

ToastyKey exposes an MCP server that Claude Code (and any MCP-compatible tool) can connect to. This allows users to query costs, manage budgets, and receive alerts without leaving their coding session.

5.1 MCP Tools Exposed

Tool Name	What It Does	Example Usage
get_spend_summary	Returns current spend for today/week/month	"How much have I spent today?"
get_project_cost	Returns cost for a specific project	"What has the Spazi project cost me?"
get_session_cost	Returns cost for the current or specified session	"How much did this session cost?"
set_budget	Creates or updates a budget cap	"Set my daily limit to 500 rupees"
get_budget_status	Returns remaining budget and % used	"Am I close to my limit?"
list_keys	Lists all stored API keys (names only, not values)	"What API keys do I have?"
add_key	Stores a new API key in the vault	"Store this OpenAI key"
get_anomaly_log	Returns recent anomaly trigger events	"Any issues with my APIs today?"
get_provider_stats	Returns usage and cost stats for a provider	"How much am I spending on ElevenLabs?"
get_cost_breakdown	Returns detailed breakdown by provider/project	"Break down my costs this week"
pause_provider	Temporarily blocks calls to a provider	"Pause all OpenAI calls"
resume_provider	Unblocks a paused provider	"Resume OpenAI"
get_recommendations	Returns cost-saving suggestions	"How can I reduce my API spend?"

5.2 MCP Resources

ToastyKey also exposes MCP resources that provide context to Claude Code without explicit tool calls: current budget status (auto-injected into context when approaching limits), active anomaly alerts, and project cost summary for the current working directory.

06. Dashboard Design

The dashboard runs on localhost:3000 and auto-updates in real-time via WebSocket. Dark theme by default (matching the dev tool aesthetic). The dashboard consists of five views:

6.1 Overview (Home)

Top bar: Total spend today (large number, with Δ vs yesterday) | Total spend this month | Active projects count | Active keys count.

Section 1 — Spend chart: Area chart showing daily spend for the last 30 days. Hover to see breakdown by provider. Toggle between USD and INR.

Section 2 — Provider breakdown: Horizontal bar chart. Each provider as a bar with spend amount. Colour-coded by provider brand colours (OpenAI = green, Anthropic = orange, ElevenLabs = purple, etc.).

Section 3 — Tangible output summary: Card grid showing human-readable outputs. “47 images generated (₹182)” “12 min audio synthesised (₹340)” “1,247 LLM calls (₹890)” “3 videos generated (₹420)”.

Section 4 — Budget status: Progress bars for each active budget. Green when under 60%, amber at 60–80%, red at 80%+.

Section 5 — Recent activity: Live feed of last 20 API calls. Each shows: time, provider icon, endpoint, cost, project name, status (success/error).

6.2 Projects View

Grid of project cards. Each card shows: project name, directory path, total cost (lifetime), cost this month, last active, number of API calls. Click to drill into per-project detail: cost breakdown by provider, cost over time chart, session list, most expensive calls.

6.3 Key Vault View

Table of all stored keys. Columns: provider icon + name, key label, status badge (active/expired/unused), last used date, monthly cost, actions (reveal/copy/edit/delete). Keys are masked by default (showing only last 4 characters). Click “Reveal” to show full key (auto-hides after 10 seconds). Import button: upload a .env file and ToastyKey parses and stores all keys found.

6.4 Triggers View

Configuration interface for anomaly triggers. Split into tabs: Global | Per Provider | Per Project. Each tab shows active triggers with: type, threshold, action, enabled toggle. Add new trigger button. Below: event log showing all trigger fires with timestamp, cause, action taken, and whether it was overridden.

6.5 Reports View

List of generated reports with: period covered, generation date, summary line (“Spent ₹4,200 across 5 providers, up 12% from last week”). Click to open full report. Generate report button for on-demand reports. Export as PDF. Configure auto-generation schedule (daily/weekly/monthly).

07. Competitor Analysis

The API monitoring space has several players, but none built specifically for the AI-native builder persona that ToastyKey targets.

Competitor	What They Do	Gap ToastyKey Fills
Helicone	Open-source LLM proxy. Logs requests, tracks costs. Cloud-hosted.	Cloud-only (data leaves your machine). LLM-specific (no ElevenLabs, no image gen). No key vault. No anomaly triggers. Developer-first UX.
Portkey	AI gateway with routing, caching, fallbacks, budget controls. YC-backed.	Cloud-hosted. Enterprise pricing ($). No MCP integration. No human-readable cost translation. No local-first option.
LiteLLM	Open-source proxy for 100+ LLMs. Budget management, spend tracking.	LLM-only (no audio, image, video APIs). Complex setup. No dashboard. No key vault. No anomaly detection.
Unkey	API key management with rate limiting and analytics.	Key management only. No cost tracking. No proxy. No budget caps. No AI-specific features.
Martian	Model router that picks cheapest model per request.	Routing only. No monitoring. No budget control. No multi-provider support beyond LLMs.
Provider dashboards	Each provider has their own usage page.	Fragmented across 10+ dashboards. No unified view. No cross-provider analytics. No budget enforcement.

> *ToastyKey's differentiation: local-first (data never leaves your machine), multi-provider (not just LLMs), MCP-native (works inside Claude Code), human-readable (non-technical builders can understand their costs), and zero-config (one install, immediate value).*

08. Branding Guide

8.1 Name & Tagline

Name: ToastyKey

Tagline: “The API cost layer for AI-native builders.”

Secondary tagline: “Track. Control. Understand.”

8.2 Visual Identity

Aesthetic: Dark mode dev tool. Think Vercel dashboard meets terminal. Clean, dense with data, no unnecessary decoration.

Colour Palette

Name	Hex	Usage
Background	#0D0D0D	Primary background
Surface	#1E293B	Cards, panels, modals
Border	#334155	Subtle separators
Text Primary	#F1F5F9	Main text on dark bg
Text Secondary	#94A3B8	Labels, captions, metadata
Accent Green	#10B981	Success, active, primary CTA, brand accent
Accent Amber	#F59E0B	Warnings, approaching limits
Accent Red	#EF4444	Errors, budget exceeded, anomalies
Accent Blue	#3B82F6	Links, informational, charts

Typography

Headings: JetBrains Mono or Courier New (monospace, technical feel). Body: Inter or system-ui (clean, readable). Code/Data: JetBrains Mono (monospace for all numerical data, API endpoints, key names).

Logo Concept

A stylised key icon with a subtle flame/toast element. Rendered in Accent Green on dark background. The icon should work at 16px (favicon), 32px (toolbar), and full size. Minimalist, geometric, recognisable at small sizes.

8.3 Voice & Tone

Direct, slightly cheeky, technical but not intimidating. We talk like a smart friend who happens to know about APIs, not like a corporate cloud provider.

Example: “Your agent burned through ₹340 while you were sleeping. We stopped it at ₹500. You’re welcome.”

Not: “An anomalous usage pattern was detected in your API consumption metrics. Automated cost governance has been applied per your configured threshold parameters.”

09. GitHub Strategy

Goal: 100 stars in the first month. This requires a killer README, a clear value proposition, and distribution through the right communities.

9.1 Repository Structure

```
toastykey/ ├── README.md              # The star-magnet ├── LICENSE
# MIT ├── package.json ├── src/ │   ├── proxy/            # API proxy
server │   ├── dashboard/         # React dashboard │   ├── mcp/
# MCP server for Claude Code │   ├── vault/             # Key encryption +
storage │   ├── tracker/           # Cost calculation engine │   ├──
triggers/          # Anomaly detection system │   └── db/                #
SQLite schema + queries ├── pricing/               # Provider pricing data
(JSON) ├── docs/                  # Extended documentation └──
screenshots/           # Dashboard screenshots for README
```

9.2 README Structure (Star Magnet)

The README must convert a visitor to a star in under 30 seconds. Structure:

Line 1: Logo + name. Line 2: One-sentence description. Line 3: Animated GIF of the dashboard (this is critical — people star repos based on visuals). Badges: stars, license, npm version. “The Problem” section (3 sentences max). “What ToastyKey Does” section (bullet points with emojis). Quick start (3 commands: install, configure, run). Screenshot gallery (dashboard views). Feature list (linked to docs). Supported providers table. Comparison table (vs Helicone, Portkey, LiteLLM). Contributing guide link. License.

9.3 Distribution Plan (First 30 Days)

Day 1: Post to r/ClaudeAI, r/ChatGPTPro, r/webdev, r/SideProject on Reddit. Day 1: Tweet/post thread showing the dashboard GIF + “I built this because I lost ₹5,000 to a runaway agent.” Day 2–3: Post on Hacker News (“Show HN: ToastyKey — Stop your AI agents from bankrupting you”). Day 3–5: Submit to Product Hunt. Day 5–10: Get listed on awesome-claude-code, awesome-mcp-servers GitHub lists. Day 10–20: Write a dev.to / Medium post: “I tracked my AI development costs for 30 days. Here’s what I found.” Day 20–30: Reach out to YouTube creators who do Claude Code content.

10. Build Plan

Phase 1: Core (Days 1–5)

Proxy server that forwards API calls for OpenAI and Anthropic. SQLite database with core schema. Basic cost calculation from token counts. Terminal output showing real-time cost per call. MCP server with get_spend_summary and get_session_cost tools.

Milestone: You can code in Claude Code, ask “how much have I spent?” and get an answer.

Phase 2: Dashboard (Days 6–10)

React dashboard on localhost:3000. Overview page with spend chart, provider breakdown, recent activity. Project tracking (auto-detect from working directory). Real-time WebSocket updates. Key vault (add, view, delete keys).

Milestone: You can open a browser and see your full API spend with a nice UI.

Phase 3: Controls (Days 11–15)

Budget caps at all levels (global, provider, project). Anomaly detection with configurable triggers and actions. Trigger event log. Budget status in MCP (Claude Code shows warning when approaching limit). Soft warnings and hard stops.

Milestone: You can set a daily budget and ToastyKey will stop your agent before it drains your wallet.

Phase 4: Polish + Launch (Days 16–20)

Add remaining providers (ElevenLabs, Cartesia, Replicate, Stability, generic REST). Tangible output translation (“47 images, 12 min audio”). Usage reports (weekly auto-generated). Dashboard screenshots and GIF for README. README polished. npm package published. GitHub repo live. Launch distribution plan begins.

Milestone: ToastyKey is live on GitHub, installable via npm, and you’re posting the launch thread.

ToastyKey

Track. Control. Understand.

A Toasty Media Project · April 2026 · Confidential